AndroNet: Privilege-Adaptive Packet Capture and On-Device Multi-Layer Threat Detection for Android

Swastik✉, Ritik, Kamal Akhter, and Syed Misbah Uddin

Department of Computer Science and Information Technology,
Central University of Jammu, Samba 181143, Jammu and Kashmir, India
swastik362004@gmail.com, ritikrajput147@gmail.com,
kamalakhter694@gmail.com, bytesyed@gmail.com

Abstract. Network intrusion detectors must infer who is talking from addresses and ports. A phone's operating system already knows which app owns each connection, yet Android monitors mostly use that knowledge to label traffic. AndroNet puts it to work for detection. The open-source analyser captures traffic without root through a local VPN, or with root at the interface, and runs four detection layers on the handset: statistical detectors and stateful rules that count per app, payload signatures, and flow-level tests that need no decryption, including a per-app profile of TLS libraries. We evaluated it on physical phones in two frozen cycles. The first uncovered defects that unit tests had hidden and cut flagged benign packets from 33.9% to 0.022%. The second replays frozen versions over 45 third-party captures unopened until the freeze. Counting only alerts on an attack's own flows, recall rises from 7 to 11 of 24 in-scope captures, including 3 of 8 Android trojans, with 0.0019% of phone packets flagged. Switching app identity off adds two false alarms in a session and stops alerts naming the app. Through the full pipeline the phone moves 160 Mbit/s down and 173 up, level with PCAPdroid, and keeps quiet connections that PCAPdroid drops, at the cost of more CPU and memory. In 41 hours of everyday use on two phones it raised 37 alerts and hit Android 15's six-hour daily cap on capture, now fixed. The VPN path stays blind to inbound scans, which the rooted path catches on an emulator.

Keywords: Mobile network security · Intrusion detection · Android VpnService · Application identity · On-device analysis.

1 Introduction

Ask a network analyst to look at suspicious traffic and they will open Wireshark without a second thought. Ask the same of the phone in their pocket and the habit fails, because packet analysers expect interface access, raw sockets and spare CPU and memory, none of which Android gives an ordinary app.

Capture is no longer the hard part. The VpnService API [2] lets an unprivileged app route device traffic through a local tunnel [14, 20], and PCAPdroid [7], the best-known open-source tool of this kind, offers that route alongside a root mode. The open question is what happens to the traffic next. PCAPdroid can flag hosts on third-party blocklists [8], while payload signatures and multi-packet behaviour remain the territory of Snort [22] and Zeek [18], which on a phone need a rooted chroot such as Kali NetHunter [16].

The phone has one thing those systems lack: it knows which app sent each packet. A network sensor only sees addresses, and inside a VPN tunnel every outgoing packet carries the same source address, so a detector counting per source ends up counting the whole phone. Counting per app keeps a browser's page load apart from a background app's burst of connections. It also lets us ask what no network sensor can: whether an app still uses the TLS library it has always used.

AndroNet is built around that advantage. Its root-free and rooted back-ends feed one pipeline of protocol dissection and four detection layers, with no server or tethered host. We evaluated it in two frozen cycles. In the first, a version that passed all of its unit tests flagged over 40% of ordinary packets the first time it ran on a phone. In the second, the corrected and extended system faced third-party captures that no development decision had seen, Android remote-access trojans among them. Our contributions are:

1. **App-attributed detection:** rate windows, thresholds and beacon tracking kept per app, and a per-app profile of TLS libraries (Sect. 5), with the value of app identity measured by switching it off (Sect. 6.4).
2. **Detection for traffic it cannot read:** tests on how a flow opens, when it talks and what DNS says about it (Sect. 5.3).
3. **One analysis path over two capture back-ends, at relay speed,** with a bounded queue between relay and detection (Sect. 4).
4. **A measured evaluation on physical phones:** frozen versions on unseen captures, cost per core class, throughput and energy against PCAPdroid, and everyday use (Sect. 6). Code, harness and results are public [26].

2 Background and Related Work

2.1 Capture and Detection

Android apps cannot open raw sockets, so the usual workaround declares the app a VPN, reads outbound traffic from a TUN descriptor and relays it through sockets the app owns, as Lumen [20] and AntMonitor [14] do. Inbound header fields are then synthesised, and only connections the device itself starts enter the tunnel [8]. PCAPdroid [7], the system closest to ours, relays with the zdtun library [9] that AndroNet also uses, classifies flows, can decrypt TLS through a mitmproxy add-on, and analyses rooted captures through the same interface.

Snort [22] set the pattern for signature matching, and Bro, now Zeek [18], for stateful analysis of network events. Attacks are rare, so the false alarm rate largely decides whether a detector is usable [3], which is why we report false alarms on real phone traffic and kept third-party captures out of development.

2.2 Detection on the Phone

AndroIDS [23] brought Snort-style signatures to Android in 2013 through an iptables queue, which needs root, and published no measurements. Chen et al. [6] built an intrusion prevention system on VpnService and tcpdump, but we could find neither an open implementation nor cost and energy figures. Commercial mobile security products are

closed, and we found no accuracy or cost data for them. Small open prototypes such as Syrrak [21] apply rate and flow rules to the headers of VpnService traffic, apparently without evaluation. HydraDragonAV Mobile [12] comes closest to our use of app identity. It attributes connections to apps and scores them from blocklists, permissions and DNS behaviour, though its documentation mentions no payload, rate or TLS fingerprint detection and gives no accuracy figures. A 2026 study trains classifiers for an app derived from PCAPdroid, tests them on a public dataset, and leaves on-device latency and energy to future work [13].

2.3 App Identity and Encrypted Traffic

Razaghpanah et al. [20] used Lumen to fingerprint the TLS handshakes of 7,258 Android apps and found that about 84% rely on the platform's default TLS API, in a study aimed at measurement alone. Matoušek et al. [15] found JA3 [1] hashes insufficient to identify a mobile app from the network, though fingerprints stayed fairly stable over time. Both results shaped our design. Identifying the app is the hard part for a network observer, and on the handset it comes for free, which leaves the easier and more useful question of whether a known app has changed. For traffic with no recognisable handshake, Wu et al. [27] described how the Great Firewall of China spots fully encrypted flows from their first packet, and we turn their exemptions into a detector.

3 Threat Model and Design Goals

AndroNet runs on the device whose traffic it watches, operated by its owner or an authorised assessor. It raises alerts and blocks nothing. The adversaries we consider leave traces in the device's own traffic: a malicious app or implant beaconing to command-and-control servers or exfiltrating data, an attacker on the same network segment who scans, floods or tampers with ARP, and hosts showing recognisable attack tooling. An adversary with kernel-level control of the device can defeat any analyser running on it, so that case is out of scope.

Four goals shaped the design: **G1**, useful analysis on unmodified consumer devices, with root adding visibility; **G2**, nothing above the capture tier branching on capture mode, only on whether a record names an app; **G3**, detection on the handset, within a per-packet budget and never holding up forwarding; **G4**, output that desktop tools can read.

4 System Architecture

AndroNet is split into three tiers: a Flutter interface, a Kotlin service layer and a C native layer (Fig. 1). On launch it checks for a working `su`. Rooted devices take the root path and all others the VPN path, though the user can override the choice. Both paths build the same record, defined in a single source file, and pass it to one pipeline.

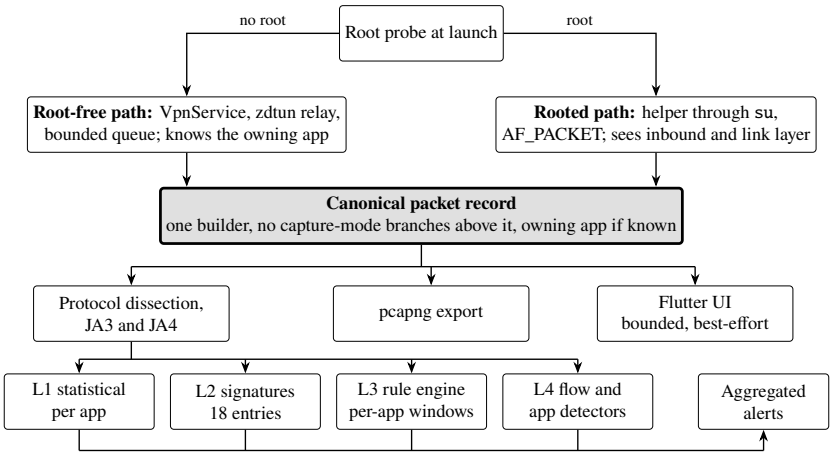


Fig. 1. AndroNet's architecture. The canonical packet record is the abstraction boundary: nothing below it knows which capture path produced a packet, only whether the record names an app.

4.1 The Packet Record and the Capture Paths

Every captured packet becomes one record: addresses and ports, protocols, size and capture timestamp, TCP flags, capture mode and direction, the owning app when the platform can resolve it, and, for ARP, the sender's address and MAC. On Android 10 and later the VPN path asks `ConnectivityManager` for the UID behind each connection and caches the app's label. The capture mode travels with the record for provenance only. No analysis component reads it, and a unit test checks that both modes raise identical alerts for identical bytes.

Root-free path. AndroNet declares a VpnService, obtains a TUN descriptor and relays traffic through zdtun [9], needing only the VPN permission the user grants. zdtun is not thread-safe, so a single loop owns it, draining the TUN descriptor and servicing its sockets. **Rooted path.** Even on a rooted device an app process cannot open a packet socket, so AndroNet launches a small helper through `su`. The helper opens one AF_PACKET socket, drops to the app's user ID (the open socket survives the switch) and streams frames over standard output. It quits as soon as the app stops reading, so a crashed app leaves no capture process behind. Either way, AndroNet can export what it analyses as pcapng that Wireshark opens directly (checked on a 3,864-packet export from the Pixel 8).

4.2 Keeping Detection off the Relay

Forwarding must never wait for detection. The relay drops each packet into a bounded queue (8192 packets or 32 MiB) and moves on, and a second thread analyses it using the capture timestamp, so rate windows follow traffic time and ignore queueing delay. A packet that finds the queue full is forwarded without analysis. Those skips are counted and raise an alert, since otherwise an attacker could blind the analyser simply by flooding

it. Payload inspection stops after 64 KiB per direction of a cleartext flow and 8 KiB of an encrypted one, the trade Suricata makes with stream depth and encryption handling [17], while headers, rates and sizes keep feeding detection and export still gets whole packets. The tunnel MTU is 10,000 bytes, as in PCAPdroid, because zdtun forwards one segment per pass of its loop, and the interface stream pauses while the app is in the background. Left alone, zdtun resets a TCP connection after 60 s of silence and breaks the connections phone apps keep open for push messages, so version 1.5.1 builds it with the idle limits RFC 4787 and RFC 5382 ask of a NAT, and room for 512 sockets.

4.3 Dissection and DNS Visibility

The dissection stage reads HTTP on any port (request line, percent-decoded URL, status, headers), DNS (question, record types, answers, response code) and TLS on any port (version, Server Name Indication, and JA3 [1] and JA4 [11] fingerprints). Hellos split across segments are reassembled up to 16 KiB, with no key material involved. Once GREASE values that Wireshark 4.6.5 keeps in signature algorithms are removed, both fingerprints match Wireshark's on 4,426 of 4,426 hellos from phone and enterprise captures.

Android moves its resolver to DNS over TLS or HTTPS whenever the server supports it, and names then vanish; on the Pixel 8 not one plain DNS packet crossed the tunnel. AndroNet refuses those encrypted connections inside the tunnel, so the resolver falls back to plain DNS that detection can read. The price is that DNS leaves the phone unencrypted while capture runs. A user who chose a strict Private DNS server keeps it, since Android has no fallback in that mode, and the DNS detectors then see nothing.

5 Four-Layer Detection Engine

Every enriched packet passes through four independent layers. Detectors count per *principal*: the app where the record names one, otherwise the source address, as on the rooted path. All state is bounded. Findings are aggregated, so repeats of one finding about one principal within 10 s yield a single alert, as with event filtering in Snort, and each alert names the app where known.

5.1 Layer 1: Statistical Detectors

Table 1 lists the detectors. During a 60 s learning period, with alerts muted, each counter's per-second peak is recorded. The threshold is then set to 2.5 times the 90th percentile of those peaks, clamped between a floor and a ceiling, where the ceiling is the larger of the default and four times the learned baseline. From then on an exponential moving average ($\alpha = 0.05$) tracks the baseline, but a sample that reached the threshold never trains it, so an attack cannot raise the bar it is measured against. A browser may open three times as many connections as other apps, because a page load legitimately fans out, an allowance only app identity makes possible. Distinct sources are counted with a 256-register HyperLogLog sketch [10] per target, so a spoofed flood costs fixed memory however many addresses it forges.

Table 1. Layer 1 detectors. Floors and defaults come from measured benign traffic.

Detector	Trigger	Severity
Port scan	destination ports from one principal in 10 s (floor 8, default 20)	High
SYN flood	SYN packets to one target per second (floor 50, default 100)	Critical
Distributed flood	300 packets per second at a target, half from distinct sources	Critical
Connection flood	new connections per principal per second (floor, default 100)	High
DNS tunnelling	10 query names over 50 characters under one zone in 60 s	Medium
ARP spoofing	binding changes, or one MAC claims two addresses (rooted path)	Critical
Malformed TCP	SYN set together with FIN or RST	Medium
Covert channel	high entropy in ICMP echo or outgoing HTTP or DNS, for 5 packets	High+

5.2 Layers 2 and 3: Signatures and Rules

The second layer matches 18 signatures in eight categories, built from patterns on payload content, named HTTP fields, ports, addresses and DNS names, combined with AND and OR. They include Meterpreter method names, Cobalt Strike's default User-Agent, scanner and login-tool fingerprints, and the WannaCry kill-switch domains. Two entries, the Tor exit and known-C2 lists, ship with sample indicators only, so real use needs a threat feed. A 64-byte tail kept per flow lets a pattern split across two consecutive segments still match.

The third layer evaluates 10 rules built from 26 predicates and the combinators And, Or and Not. They cover horizontal scans (over 100 destinations in 10 s, 300 for browsers), SQL injection, executable downloads, DNS tunnelling, brute force, cryptomining, bulk uploads (over 1 MiB per 10 s to one destination), shellcode, scanner User-Agents and ICMP tunnelling. Evaluation takes two passes. The first ignores rate predicates, and a packet that fails it is not the kind the rule counts, so it never enters the window. A packet that passes joins a window kept per rule and per principal, and the second pass evaluates rates over that window alone, so "more than 10 per second" means ten suspicious packets from this app. Each rule keeps at most 1000 entries for each of 64 principals.

5.3 Layer 4: Flow and App Detectors

Most phone traffic is encrypted, which leaves the layers above little to read. The fourth layer works from what any flow reveals anyway: which library opened it, what its first bytes look like, when it talks, and what DNS says about it (Table 2).

TLS library profile. Full JA4 values turned out too volatile to profile. In one phone session, 27 distinct values came from 4 cipher lists, because ALPN, the server name and resumption change the rest of the fingerprint from one connection to the next. AndroNet therefore profiles each app by the part of JA4 that hashes its cipher list. An app counts as settled after 20 consecutive hellos with no new list, and a new list from a settled app raises one alert and restarts settling. Profiles hold 16 lists for each of 256 apps and survive restarts. An implant that brings its own TLS stack into an app shows up here, and so does a plugin or a major library update, the false alarm this test accepts.

Opaque first payloads. We test a flow's first payload against the exemptions Wu et al. [27] documented: bits set per byte outside 3.4 to 4.6, six printable bytes at the start,

Table 2. Layer 4 detectors. All need no decryption, and APP-001 needs app identity. FLOW-001 to FLOW-003 report at most once an hour per principal and target.

Detector	Trigger
APP-001 new TLS library	a settled app presents a cipher list new to it
FLOW-001 opaque flow	first payload looks uniformly random, on a port with no expected encryption
FLOW-002 protocol mismatch	first payload on port 22, 53 or 80 does not speak that port's protocol
FLOW-003 beacon	gaps between bursts are regular, to a destination with no DNS or TLS name seen
FLOW-004 TXT channel	3 TXT answers of 100 bytes or more under one zone in 60 s
FLOW-005 failed lookups	20 distinct names fail to resolve for one principal in 60 s

over half the bytes printable, a printable run over 20 bytes, or a recognisable TLS or HTTP start. A payload that earns none of these raises an alert unless its port is known to carry encryption. Port 5222 is excused too, after a live run showed WhatsApp's own encrypted protocol there.

Mismatch and beacons. Traffic on ports 22, 53 and 80 should open with an SSH banner, a DNS message or an HTTP method. For beacons we keep the last 16 gaps between activity bursts for each principal and destination. A flow looks like a timer when at least 12 gaps exist, their median lies between 10 s and 10 minutes, and three quarters sit within 10% of the median. It raises an alert only if no DNS or TLS name has been seen for the destination, so push services, which have one, stay quiet. Using the median keeps a single long pause from hiding a timer.

6 Evaluation

We ask six questions. How accurate is detection on traffic no development decision saw? What does app identity add? Do the two capture paths behave alike? What does analysis cost on real hardware? How does the whole system compare with PCAPdroid? And how does it behave in everyday use?

6.1 Setup and Data

Hardware. Our main phone is a Pixel 8 (Tensor G3, 8 GB RAM, Android 17, unrooted user build) on 802.11ax Wi-Fi, sharing an access point with the lab laptop. A second unrooted phone, a mid-range Samsung Galaxy M53 (MediaTek MT6877, 6 GB RAM, Android 16), ran the live scenarios, a browsing and app session, and shorter speed tests. Stock user builds cannot exercise the rooted path, so we ran it on an Android 16 x86-64 emulator image with root.

Harness. Replay experiments run the production Kotlin classes under ART on the phone, through `app_process`, with the clock following trace time. Versions 1.5.0 and 1.5.1 each gave identical verdicts on the phone and a desktop JVM for all 47 traces of the second cycle (1,514,251 packets), even with different regular expression engines.

Table 3. Evaluation data. The source of every capture is listed in the repository [26].

Set	Traffic	Captures	Packets
First cycle	third-party attack captures	28	108,168
(v1.5.0 development)	third-party benign captures, including bigFlows [25]	11	810,700
	Pixel 8 browsing, after the first freeze	1	201,020
Second cycle	Android remote-access trojans	8	563,334
(unopened)	floods, and a backdoor using domain generation	16	624,791
	Windows-domain techniques, no detector targets	21	6,890
	phone sessions after the freeze, browsing and apps	2	319,236

Live experiments use a release build that differs from a normal one only in accepting scripted commands. Unless stated otherwise, measurements on the phone use version 1.5.1.

Data. Table 3 summarises the two cycles. In the first, we fixed a split, froze version 1.4.0 with file hashes recorded [26], and only then replayed third-party captures labelled from their publishers' descriptions. New detectors can only be validated on new data, so that set then became development data for version 1.5.0. For the second cycle we chose captures by a fixed rule from names and descriptions, opening none: all eight remote-access trojans in Android Mischief [4], three recorded on a physical phone and five on emulators; every StopDDoS flood capture [24] up to 50 MB that the first cycle left unused; and every unused Bousseaden capture [5], most showing Windows-domain techniques no detector targets, which we keep and score apart. We wrote the expected alerts first, recorded the captures' hashes, froze both versions [26], and replayed them on the same data.

6.2 What Measurement Changed

Version 1.3.0 passed all 106 of its unit tests, yet its first measurement exposed, among others, a dissector and detectors that disagreed on field names, so URL signatures never fired; thresholds sampled per packet while counters still rose, so a 400 pps SYN flood raised no alert; rate rules counting all traffic from all hosts, with 95,673 false alerts on one phone session; quadratic regular expressions under ICU, at 7.7 ms per packet; and zdtun driven from two coroutines at once, which held the tunnel to 15 Mbit/s. The unit tests built their input records by hand, so stages could each pass while disagreeing. We now push real wire bytes through the whole pipeline in a contract test suite, part of the 208 tests that version 1.5.1 passes.

Real phones then corrected version 1.5.0 before its freeze. WhatsApp's first payload on port 5222 looks fully encrypted. Full JA4 values changed within one TLS library. ICMP errors quoting QUIC packets resembled a covert channel. Advertising hosts tripped a DNS rule that fired on single long names, and repeated lookups of the same hosts looked like domain generation. A page's burst of large post-quantum TLS hellos read as a bulk upload. Each case led to a narrower test, and no lab trace had shown any of them.

Table 4. Detection by version. The first cycle's set was development data for v1.5.0, so only the second cycle is an unseen test of it. v1.5.1 fixes one defect that the second cycle exposed, so neither set is an unseen test of that version.

Measure	v1.3.0	v1.4.0	v1.5.0	v1.5.1
First cycle: 1.01 million benign packets				
Capture recall (in scope)	12/19	12/19	17/19	17/19
Packet false positive rate	33.9%	0.022%	0.021%	0.021%
Alerts on benign traffic	352,310	218	215	213
Phone browsing: packets flagged	48.8%	0.001%	0.000%	0.000%
Second cycle: captures unopened until the freeze				
Capture recall, by the rule fixed in advance		7/24	14/24	11/24
Recall on the attack's own flows		7/24	11/24	11/24
Android remote-access trojans		1/8	3/8	3/8
Floods		6/15	8/15	8/15
Domain-generating backdoor		0/1	0/1	0/1
Alerts off the attack's flows		99	15	4
Alerts on untargeted captures		0	0	0
Phone sessions: alerts		33	6	5
Phone sessions: packets flagged		0.0103%	0.0019%	0.0016%
Alert precision		82.8%	96.8%	98.6%

6.3 Detection Accuracy

We report *capture recall*, the share of in-scope attack captures that raise at least one expected alert, and the *packet false positive rate*, the share of benign packets that raise any alert. For the second cycle we also report recall counting only alerts raised by the attack's own packets, and *alert precision*, the share of all alerts that such packets raise. Table 4 gives both cycles.

First cycle. Version 1.3.0 "detected" SSH password guessing by alerting on every packet to port 22, and raised 352,310 alerts on benign traffic. Version 1.4.0 kept its recall honestly and cut those alerts to 218. Its misses pointed to the new detectors: a spoofed flood from 9,886 addresses, ARP poisoning with no binding change, a shell on TCP port 53 and commands in DNS TXT answers. Version 1.5.0 catches these, which is development evidence only. It still misses Nmap's version scan, which touches two ports, and a Meterpreter capture that begins mid-flow and so has no first payload to test.

Second cycle. By the rule fixed in advance, version 1.5.0 detects 14 of 24 captures and version 1.4.0 detects 7. That rule accepts an expected alert anywhere in a capture, and these captures come from phones in everyday use, so we then read the packet behind every alert [26]. This check, written after the run, can only lower recall. Version 1.4.0 stays at 7, version 1.5.0 falls to 11, and we take these figures as the result. Eleven FLOW-003 alerts fell on connections to public web services, and in three captures they were the only hit. Each followed the end of a DNS entry's five-minute life: the connection outlived the entry that named its address, and the beacon test took the address for an unnamed one. Version 1.5.1 remembers a name once seen for that destination. On the same captures it raises none of those alerts and keeps all 11 detections, giving up only

one beacon alert on a controller whose address had a name, in a capture FLOW-001 also catches. These captures produced the fix, so they cannot test it; the everyday-use results below can.

Among the trojans, FLOW-001 caught the first payload to the controller in two captures, and Saefko's IRC channel raises a signature that version 1.4.0 also has. The misses show the limits of what a first payload can tell. Ahmyth and a command-line AndroRAT open with printable text, AndroRAT with a structured and largely printable message, and DroidJack's controller with a six-byte header, so none of them looks random or matches a signature, and HawkShaw's capture shows only web and messaging services, with no controller outside port 443. Among the floods, five of version 1.4.0's six hits came from its ICMP tunnel rule firing on error messages that quote the flood. Version 1.5.0 reads ICMP data only from echo messages, after the same errors raised false alarms on QUIC traffic, which cost it three of those captures while the distributed-flood detector gained five. Neither version detects the domain-generating backdoor, whose capture holds 28 packets, and neither raises any alert on the 21 Windows-domain captures. Long attacks repeat their alerts, which flatters precision in every column.

A yardstick. Suricata 8.0.7 [17] with the ET Open rules [19] (52,849 signatures, downloaded for this comparison), run on a desktop over the same captures and counting any alert on an attack's flows, reaches 14 of 24. Its rules name AhMyth and the domain-generating backdoor, and Saefko's IRC traffic raises them too. They stay silent on the controller flows of the other six trojans, two of which AndroNet's first-payload test catches, so signatures and flow tests miss different things, and together they reach 4 of 8 trojans. Of Suricata's 11 flood hits, seven come solely from IP reputation rules written years after these captures, and only one from a rule that names a denial of service, whereas AndroNet's come from rate alone. On the phone sessions it raises 56 informational alerts, none claiming an attack. Counting any alert, with rules newer than the attacks, favours Suricata, which also needs a separate host or a rooted handset.

We also ran 28 attack scenarios live on the Pixel 8 through the real tunnel, with the app's alert log as the only output. Of these, 28 raised their expected alerts, and all 44 alerts named an app: the attacking process in 43, and an unrelated app that APP-001 flagged in one. Frozen version 1.5.0 detected 27, raising no alert for failed lookups, which needs answers from the lab's DNS server. The laptop has two adapters on the network, so that server's answers could leave from an address the phone would reject. Once it answered from the address asked, the scenario fired in 10 of 10 runs, though we cannot show that this caused the miss. On the second phone, 28 did, and its browsing and app session raised no alerts over 179,730 packets. Because these scenarios resemble the development traces, this verifies the path end to end and makes no independent accuracy claim.

6.4 What App Identity Adds

We ran frozen version 1.5.0 with app identity switched off, so that detectors count per source address as a network sensor must, over two consecutive 13-minute sessions on the same sites and apps. With identity on, the live session raised no alerts over 139,506 packets. Replaying those same packets without identity raised one, and the live session with identity off raised two over 98,403 packets, all of them the horizontal scan rule

reading page loads as a scan. Of the 28 attack scenarios, version 1.5.1 detected 27 with identity off, losing only the TLS library change, which needs identity by definition. These counts are small and bound the claim: on this traffic identity spares a few false alarms and changes no other detection. Beyond that, an alert with identity names the app to remove, where without it every alert names the phone's address.

6.5 Capture-Path Parity and Observability

Running the same 27 scenarios through the rooted path gave the same expected detections, direct evidence for goal G2; the one scenario left out needs app identity. Another 7 traces, injected as real frames on the interface and covering inbound SYN, FIN, NULL and Xmas scans, a SYN flood and ARP spoofing, each raised the expected alert. The helper exited within 5 s of the app being killed. These rooted runs used an emulator and a stand-in for `su`, so the rooted path has yet to run on a physical rooted handset.

Identical analysis does not mean identical visibility. Only the VPN path knows which app sent a packet, and only the rooted path sees inbound scans and floods, ARP and other link-level traffic, and inbound headers as sent. To confirm the blind spot, we scanned the phone with real nmap from the laptop during VPN-mode capture. Over three scans of 1000 ports each, the phone's kernel answered 3,000 probes with resets, so the packets did arrive, yet AndroNet raised no scan alerts, because unsolicited inbound packets never enter a VPN tunnel. Every VPN-based tool shares this blind spot, and it is the strongest reason to keep a rooted path.

6.6 Per-Packet Cost

We cost each layer by ablation: a trace is replayed with the signature and rule lists emptied by reflection, so the production code needs no timers. Layer 4 runs inside the Layer 1 stage and is costed with it. On a Cortex-A715 mid core, replaying tcpreplay smallFlows, the rule layer dropped from 7.2 ms to 20 µs per packet. Version 1.5.1 averages 105 µs per packet (21 at the median, 336 at p95), enough for about 9,700 packets per second on one mid core, 13,700 on the big core and 2,600 on a little core. With the scheduler free to choose its core, the Samsung Galaxy M53 sustains about 8,500. After replaying the trace, detection state occupies 836 KiB of heap.

6.7 Throughput, Energy and PCAPdroid

Table 5 compares no capture, PCAPdroid 2.0.2 and AndroNet on the same phone and network, with the conditions shuffled in each of five rounds. Figures are medians of 50 MB transfers, two per direction per round. For energy, a script on the phone logged battery current and voltage once a second over 150 s, with the cable out, the screen on at minimum brightness and three repetitions in rotating order. The current comes from the fuel gauge, so these figures are coarser than an external power monitor would give. We measured PCAPdroid's free build without writing a capture file, and did not test its paid blocklist feature.

The relay is no longer what limits the phone. With the full app in the background it moves 160 Mbit/s down and 173 up, over three times version 1.4.0's 39 and 47, and

Table 5. Head-to-head on the Pixel 8. Throughput and connect time are TCP between phone and laptop. AndroNet 1.4.0, measured in an earlier session, ran its full app with a 1500-byte tunnel. Version 1.5.1 is shown forwarding alone, and as the full app with its interface in front or in the background. Power is whole-device, under a 10 Mbit/s download and at idle.

Measure	None	PCAPdroid	v1.4.0	v1.5.1		
			full	forward	in front	in back
Download (Mbit/s)	181	183	39	163	177	160
Upload (Mbit/s)	158	153	47	180	163	173
TCP connect (ms)	24.6	38.1	30.0	28.6	34.9	34.3
App CPU (% of a core)	–	19	128	31	71	66
Peak memory (MiB)	–	151	414	258	316	311
Power, 10 Mbit/s (W)	1.27	1.34	1.59			1.26
Power, idle (W)	0.94	0.99	0.96			1.09
Battery, 10 Mbit/s (%/h)	7.8	8.1	9.4			8.2

Table 6. Capabilities compared, from each tool's documentation and our measurements. • present, ○ partial or conditional, – absent.

Capability	Wireshark	Snort	PCAPdroid	AndroNet
Runs on an unrooted Android phone	–	–	•	•
Names the app behind each connection	–	–	•	•
Attack detection that uses the app's identity	–	–	–	•
TLS decryption	○[a]	–	•	–
Reputation blocklists	–	•	○[b]	○[c]
Payload signatures and stateful rules	–	•	–	•
Volumetric attack detection	–	○	–	•
Connection blocking	–	•	○[b]	–

[a] with supplied keys. [b] paid feature. [c] sample lists only.

level with PCAPdroid given the link, since single transfers with no capture ranged from 110 to 200 Mbit/s. The analysis queue skipped no packets in any round. What this costs is processor and memory, 66% of a core and 311 MiB against PCAPdroid's 19% and 151 MiB, and it buys detection that PCAPdroid's free build does not run. The Samsung Galaxy M53 moved 86 Mbit/s down and 127 up through the full app over two rounds, against 122 and 131 with no capture. Quiet connections separate the relays. We opened a TCP connection through each tool, stayed silent, then sent a line and waited for its echo. With no capture and through version 1.5.1 the connection survives 300 s of silence, and 600 s on the emulator. Through version 1.5.0 and PCAPdroid, which also relays with zdtun, it is dead by 90 s, and an app holding one open for push messages has to reconnect. Under the 10 Mbit/s download both apps' draw stays within the spread between repetitions of the no-capture baseline, with AndroNet's process at 21% of a core against PCAPdroid's 5%. At idle AndroNet adds 0.15 W, while PCAPdroid's draw stays within the spread.

PCAPdroid remains the broader capture and visibility tool, with richer classification, TLS decryption and paid blocking (Table 6). What AndroNet adds is detection: signatures, stateful rules, volumetric detectors and flow tests together, tied to the app that caused them, on an unrooted handset that stays online, in open source, and measured on that handset. Each of these has precedents (Sect. 2), and we found no other system that combines all of them.

6.8 Everyday Use

Two phones ran version 1.5.1 in their owners' everyday use, with profiles empty at the start and logging only each alert's time, finding and app: the Pixel 8, an author's own phone, and an entry-level Oppo A53 (Snapdragon 460, 4 GB RAM, Android 12). On the Pixel, capture ended after 11 hours, once the app had run for six: Android 15 and later allow a data-sync foreground service, the type version 1.5.1 declared, six hours of running time a day and crash an app that keeps one running longer. Version 1.5.2 declares a special-use service instead and changes nothing else; with the limit cut to five minutes, 1.5.1 crashed again and 1.5.2 kept its tunnel up for all eight minutes. On the Oppo, capture ran for 30 hours in one process, until the phone restarted to install a system update, with no crash or freeze recorded. They analysed 0.6 and 0.3 million packets with none skipped. The Pixel raised 16 alerts, about 34.6 a day, and the Oppo 21, about 17.0 a day, each naming an app: 16 from FLOW-003, 12 from the bulk upload rule and nine from APP-001. The Oppo's 16 FLOW-003 alerts all came from one app, about once an hour. No attack was staged, so we count every alert as a false alarm.

6.9 Threats to Validity

Our unseen attack set is small, so recall carries wide uncertainty, which is why we publish per-capture results. We named each trojan's controller by reading its capture, and that reading is the one judgement call in the flow check. Five of the eight trojan captures come from emulators, and replay carries no app identity, so APP-001 and the per-app windows are tested only in live runs and phone sessions. Most other third-party traces come from desktop networks. The replayed phone sessions come from two phones on Wi-Fi. Limits were set on one phone, and a session on one other is thin evidence that they suit others. Wi-Fi capacity also varied from round to round, so small throughput differences mean little.

7 Discussion and Limitations

What an app can do to stay unseen. A malicious app that speaks TLS on port 443 through the platform's default library, to a host with a DNS name, at irregular times, shows little to any layer except volume. An implant could also use an excused port such as 5222, pad its first payload to look printable, or jitter its timer beyond the 10% band. AndroNet keeps encrypted payloads out of matching and adds no certificate of its own, which leaves only what such a flow has to reveal: rates, sizes, names and its handshake.

What detection gives up for speed. A pattern beyond the inspection depth escapes matching, the 64-byte tail falls short of stream reassembly, and a sustained flood can force the queue to skip packets, though it reports doing so. Refusing encrypted DNS exposes names on the local path during capture, a cost users should weigh. Future work includes both capture paths on a physical rooted phone, threat feeds and flow blocking.

8 Conclusion

AndroNet takes packet capture on Android through to detection on the handset, using what only a handset knows: which app sent the packet. Measured on physical phones and on captures unopened until the code was frozen, and counting only alerts on the attack's own flows, it detects 11 of 24 in-scope attacks, including 3 of 8 Android remote-access trojans, where its predecessor detected 7. Its relay keeps pace with PCAPdroid's and holds quiet connections that PCAPdroid drops, while using more CPU and memory. The evaluation also makes its limits plain: the misses listed above, a blind spot for inbound traffic on the VPN path, and a rooted path still waiting for physical hardware.

Acknowledgments. We thank the anonymous reviewers for their helpful comments, our department for its support, and the authors of the open-source tools and public datasets this work builds on. Claude (Anthropic) assisted the authors throughout this work: it drafted and revised the text, including the survey of related work, wrote the evaluation and analysis code, implemented the code changes and detectors described here, and helped run the experiments. All results come from running the published code. The authors reviewed the text and code, verified the results, and take full responsibility for this publication.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Althouse, J., Atkinson, J., Atkins, J.: JA3: A method for profiling SSL/TLS clients. https://github.com/salesforce/ja3, last accessed 2026/11/03
2. Android Developers: VpnService API reference. https://developer.android.com/reference/android/net/VpnService, last accessed 2026/11/02
3. Axelsson, S.: The base-rate fallacy and the difficulty of intrusion detection. ACM Transactions on Information and System Security **3**(3), 186–205 (2000). https://doi.org/10.1145/357830.357849
4. Babayeva, K.: Android Mischief dataset. Mendeley Data, version 1 (2021), https://doi.org/10.17632/xbx2j63xfd.1
5. Bousseaden, S.: PCAP-ATTACK: Samples of post-exploitation techniques. https://github.com/sbousseaden/PCAP-ATTACK, last accessed 2026/11/02
6. Chen, G., Zhou, K., Peng, Y., Zhou, L., Zhang, Y.: A novel network intrusion prevention system based on Android platform. International Journal of Internet Protocol Technology **14**(2), 67–76 (2021). https://doi.org/10.1504/IJIPT.2021.116254
7. Faranda, E.: PCAPdroid: No-root network monitor, firewall and PCAP dumper for Android. Version 2.0.2, https://github.com/emanuele-f/PCAPdroid, last accessed 2026/11/02

8. Faranda, E.: PCAPdroid user guide. https://emanuele-f.github.io/PCAPdroid/, last accessed 2026/11/02
9. Faranda, E.: zdtun: Lightweight TCP/UDP/ICMP tunnelling library. https://github.com/emanuele-f/zdtun, last accessed 2026/11/02
10. Flajolet, P., Fusy, É., Gandouet, O., Meunier, F.: HyperLogLog: The analysis of a near-optimal cardinality estimation algorithm. In: 2007 Conference on Analysis of Algorithms (AofA 07). DMTCS Proceedings, vol. AH, pp. 127–146 (2007). https://doi.org/10.46298/dmtcs.3545
11. FoxIO: JA4+ network fingerprinting. https://github.com/FoxIO-LLC/ja4, last accessed 2026/11/03
12. HydraDragon Antivirus: HydraDragonAV Mobile: Network security. https://github.com/HydraDragonAntivirus/HydraDragonAV-Mobile/wiki/Network-Security, last accessed 2026/11/03
13. Khalemskii, N.D., Beschastnyi, V.A.: Development of the malicious traffic detection system for mobile applications security. Discrete and Continuous Models and Applied Computational Science **34**(2), 187–200 (2026)
14. Le, A., Varmarken, J., Langhoff, S., Shuba, A., Gjoka, M., Markopoulou, A.: AntMonitor: A system for monitoring from mobile devices. In: 2015 ACM SIGCOMM Workshop on Crowdsourcing and Crowdsharing of Big (Internet) Data. pp. 15–20. ACM (2015). https://doi.org/10.1145/2787394.2787396
15. Matoušek, P., Burgetová, I., Ryšavý, O., Victor, M.: On reliability of JA3 hashes for fingerprinting mobile applications. In: Digital Forensics and Cyber Crime (ICDF2C 2020). LNICST, vol. 351, pp. 1–22. Springer (2021). https://doi.org/10.1007/978-3-030-68734-2_1
16. OffSec: Kali NetHunter documentation. https://www.kali.org/docs/nethunter/, last accessed 2026/11/02
17. Open Information Security Foundation: Suricata user guide. https://docs.suricata.io/, last accessed 2026/11/03
18. Paxson, V.: Bro: A system for detecting network intruders in real-time. Computer Networks **31**(23–24), 2435–2463 (1999). https://doi.org/10.1016/S1389-1286(99)00112-7
19. Proofpoint: Emerging Threats Open ruleset. https://rules.emergingthreats.net/open/, last accessed 2026/11/04
20. Razaghpanah, A., Niaki, A.A., Vallina-Rodriguez, N., Sundaresan, S., Amann, J., Gill, P.: Studying TLS usage in Android apps. In: 13th International Conference on Emerging Networking Experiments and Technologies (CoNEXT '17). pp. 350–362. ACM (2017). https://doi.org/10.1145/3143361.3143400
21. rcallaby: Syrrak: On-device mobile intrusion detection system for Android. https://github.com/rcallaby/Syrrak, last accessed 2026/11/03
22. Roesch, M.: Snort: Lightweight intrusion detection for networks. In: 13th USENIX Systems Administration Conference (LISA '99). pp. 229–238. USENIX Association (1999)
23. Sánchez, J.: Building the first Android IDS on network level. DeepSec 2013, slides, https://deepsec.net/docs/Slides/2013/DeepSec_2013_Jaime_Sanchez_-_Building_The_First_Android_IDS_On_Network_Level.pdf, last accessed 2026/11/03
24. StopDDoS: Packet captures of DDoS attacks. https://github.com/StopDDoS/packet-captures, last accessed 2026/11/02
25. Tcpreplay project: Sample captures. https://tcpreplay.appneta.com/wiki/captures.html, last accessed 2026/11/02
26. The AndroNet authors: AndroNet: Source code, evaluation harness and result files. Version 1.5.2, https://github.com/amibhai/AndroNet, last accessed 2026/11/09
27. Wu, M., Sippe, J., Sivakumar, D., Burg, J., Anderson, P., Wang, X., Bock, K., Houmansadr, A., Levin, D., Wustrow, E.: How the Great Firewall of China detects and blocks fully encrypted traffic. In: 32nd USENIX Security Symposium. pp. 2653–2670. USENIX Association (2023)